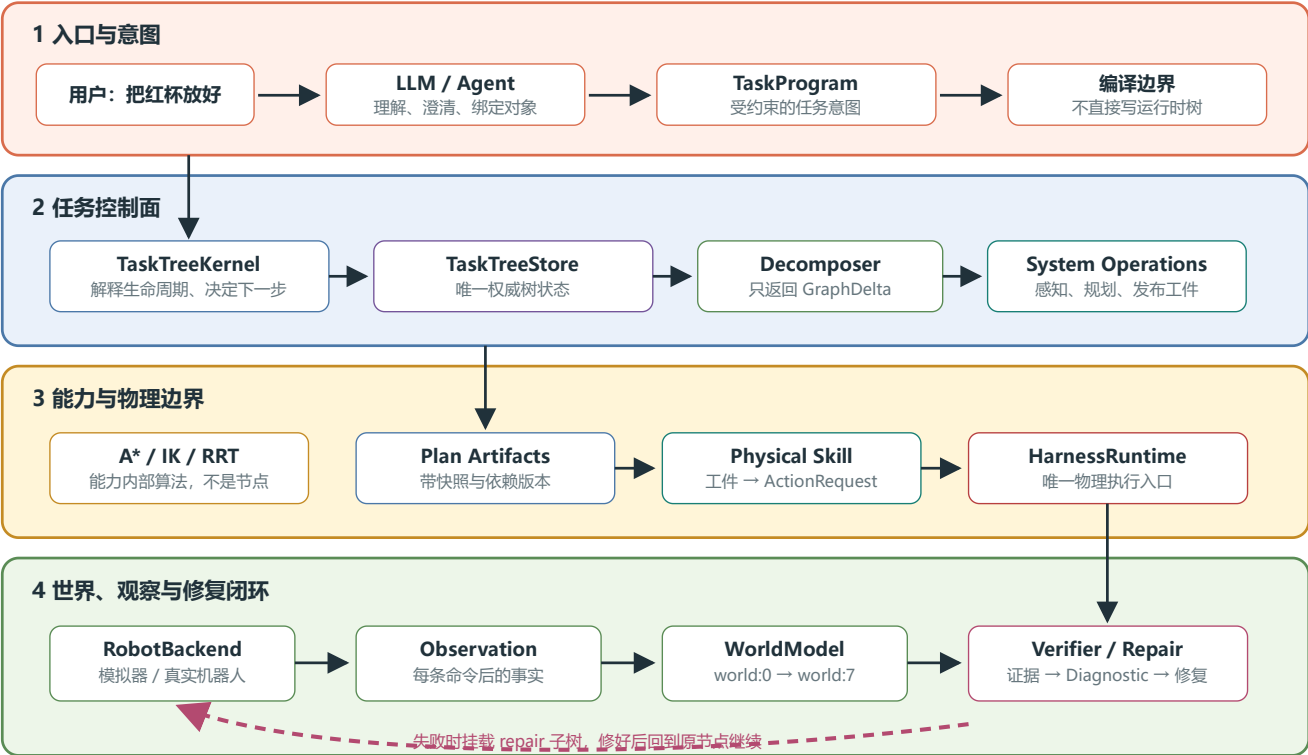


把 Agent 框架 讲到人人都能看懂

从一句“把红杯放好”，一路讲到任务树、规划工件、机器人动作、观察证据，以及失败后的递归修复。

白话 结构图 真实案例 故障修复



先记住一句话： LLM 负责理解目标，Kernel 负责控制过程，Harness 负责承载真实世界，Repair 负责把失败重新变成工作。

先给最终结论：这套系统到底是什么？

如果只允许你记住一页，就记住下面这条主线。

一句人话

它不是“让大模型直接控制机器人”，而是给 Agent 加了一层可检查、可追踪、可修复的执行控制面。

1 理解 把人话变成明确目标	2 拆解 把目标变成任务树	3 执行 把计划变成受控动作	4 恢复 发现失败就挂修复树
---	--	---	---

老板关心的结果

- 出了问题能说清楚：哪一步、为什么、影响什么。
- 不是每次失败都人工从头开始，能局部修复。
- 换算法、换机器人时，业务任务不用全部重写。
- 连续任务能保持真实现场，不会每次都把世界重置。

CTO 关心的机制

- TaskTreeStore 是唯一权威运行时树状态。
- TaskTreeKernel 是唯一节点生命周期解释器和 Store 写入者。
- LLM 只能提交规范化 TaskProgram，不能写树或调机器人。
- HarnessRuntime 是唯一物理执行入口，每条命令后都有 Observation。

读图方法

后面每一页都回答一个具体问题：谁负责？看什么证据？失败后怎么办？不要把“模型会说话”误认为“系统已经可靠执行”。

第一段 第 3-7 页：先用杯子任务建立直觉	第二段 第 8-20 页：看清对象、分层、权限和生命周期	第三段 第 21-26 页：完整走一遍正常任务	第四段 第 27-32 页：完整走一遍故障与递归修复	第五段 第 33-41 页：连续会话、安全、价值和生产化
--	--	---	--	--

先别怕术语：我们真正要解决的是什么？

先用一个生活里最简单的动作，把所有复杂词汇压缩成一句话。

“请把红色杯子放到右边的投放区。”

这句话听起来只有一个动作，但机器人要面对一串现实问题。

对象是谁？

识别

红色杯子是哪个实体？现场可能有多个杯子，不能只凭名字猜。

目标在哪里？

绑定

“右边”是哪个区域？需要一个可验证的目标位置和坐标。

路线能走吗？

规划

中间有没有墙、箱子或其他障碍？拿起杯子后路线是否仍然成立？

动作真的成功吗？

验证

命令发出不等于杯子被拿起，更不等于杯子最终已经放好。

核心转折

人只说“结果”，系统必须把结果拆成一连串可检查的前置条件、动作和后置证据。

为什么不能让 LLM 直接调用机器人？

直接调用很快能跑通 Demo，但在真实世界里，最危险的恰恰是“看起来跑通了”。

黑盒方式

LLM → robot.move() → 返回 success

- 模型既决定做什么，又决定怎么做，还可能直接触发副作用。
- 失败只剩一句“没成功”，很难定位到底是路线、抓取还是验证出了问题。
- 重试可能重复抓取、重复释放，造成二次副作用。
- 过程没有统一的任务状态、快照、事务和审计边界。

控制面方式

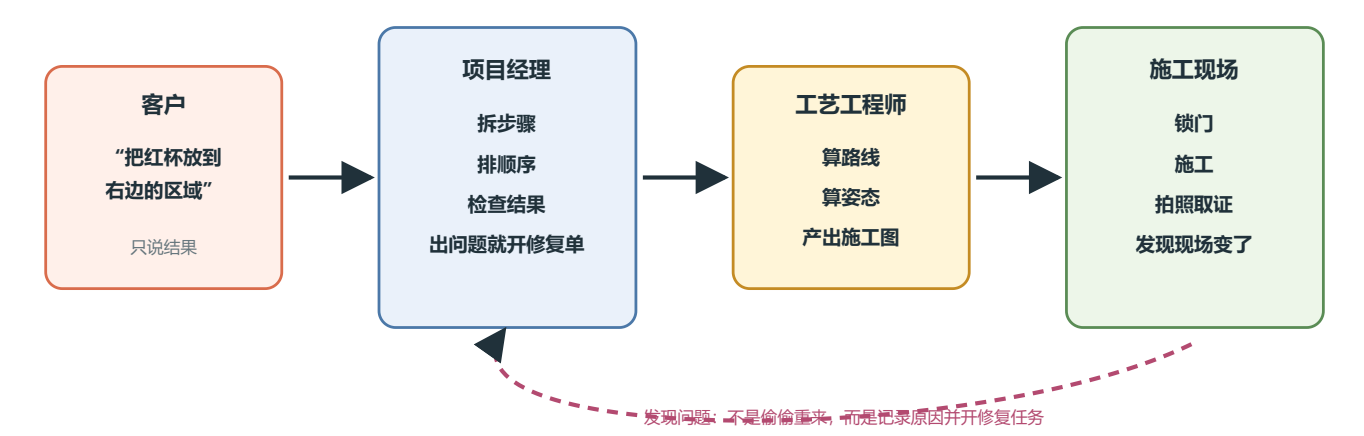
意图 → 任务树 → 工件 → ActionRequest → Runtime → Observation

- 每一步都有明确责任人和权限。
- 每次动作都能关联到任务节点、计划版本和世界快照。
- 失败先变成结构化 Diagnostic，再决定是否修复。
- 模型不能越过 Runtime 直接碰后端。

关键区别： 黑盒关注“有没有返回值”；控制面关注“现场是否真的变了，而且变成了预期的样子”。

用一座工厂来理解整套架构

这是最容易记住的类比：用户是客户，LLM 是接单员，Kernel 是项目经理，规划器是工艺工程师，Runtime 是施工总闸门。



客户	只说想要的结果：“杯子放好”。
接单员 / LLM	理解人话，确认对象，填写标准订单。
项目经理 / Kernel	安排顺序，记录状态，决定失败后下一步。
工艺工程师 / A*、IK、RRT	计算路线、姿态和运动方案，但不偷偷搬东西。
施工总闸门 / HarnessRuntime	检查现场、锁资源、执行动作、收集证据。

为什么这个类比重要？

因为它把“会思考”和“有权执行”分开了。项目经理可以决定下一步，但不应该绕过安全闸门直接开机器。

一句人话，如何变成系统能执行的任务？

系统不会把整句自然语言原样塞给机器人，而是先转换成一个受约束的任务意图。



人说的	系统必须明确的	为什么
红杯	实体 ID: cup-red	防止抓错对象
投放区	区域 ID: drop-zone	防止目标漂移
放好	位置误差、释放状态、最终验证	防止“命令成功但结果不对”

不要把它理解成形式主义

这些字段的作用是把模糊的人话变成可以检查、可以追责、可以重试的对象。

最小案例：Place 到底包含多少事情？

用户只说一个“放置”，系统内部至少要回答五类问题。

目标	把 cup-red 放到 drop-zone
准备	找到对象、确认目标、生成抓取计划
移动	先到抓取位置，再拿起，再带着物体走
完成	释放、确认已松开、确认最终位置正确
异常	路线被挡时，先修复阻碍，再回到原任务



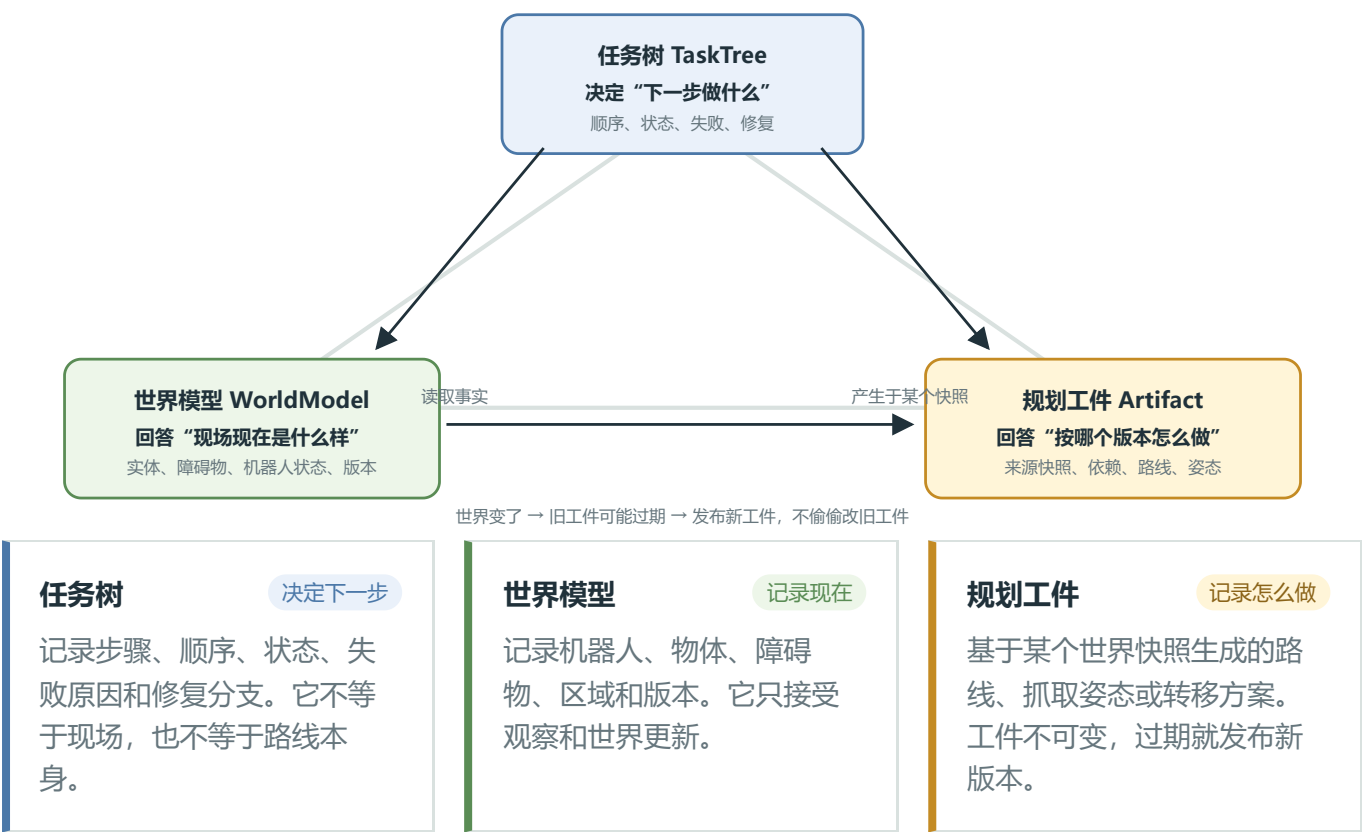
注意：这里的“+”不是数学加法，而是说明一个大目标由多个有先后关系的小工作组成。每个小工作都能独立记录成功或失败。

提前规划的原因

系统会在还没有拿起杯子时，先检查“带着杯子能不能走到目标”。如果这时就发现路不通，修复会更安全，因为夹爪还是空的。

整套架构只有三个核心对象要先记住

把这三样东西分开，后面的所有设计就不容易混乱。



最简单的记忆口诀
树管顺序，世界管事实，工件管方案。

什么是任务树？把它当成“会执行的待办清单”

普通待办清单只写要做什么；任务树还记录顺序、状态、证据和失败后的分支。



任务树不是日志，也不是一大坨 JSON

这三个东西各有职责，混在一起以后，谁拥有状态、哪个计划过期、失败在哪里都会变得不清楚。

任务树 结构 “应该按什么顺序做？现在做到哪一步？” 由 Kernel 解释，由 Store 保存。	世界快照 事实 “机器人和物体此刻在哪里？现场版本是多少？” 由 Observation 推进。	规划工件 方案 “在 world:0 的条件下，路线和姿态怎么走？” 不可变，按依赖判断新鲜度。
--	--	--

错误想象

```
{ intent, current_state, plan, result, error, retry }
```

所有东西挤在一个对象里，看似省事，实际上无法准确判断“计划为什么失效”。

正确的分工

任务树可以继续存在，世界快照可以继续推进，旧工件可以保留作审计；三者不需要互相覆盖。

机器人不是生活在模型的想象里，而是生活在一个会变化的物理世界里。

机器人	base=(6.5, 4.5), 夹爪已打开, 未持物
drop-zone	目标区域, 允许误差 0.3

map 障碍占用版本已记录



最重要的边界

规划器和 Verifier 读取 WorldSnapshot；它们不能直接读取机器人后端的私有状态，更不能绕过观察把“猜测”写成事实。

什么是 Artifact? 把它当成 “带版本的施工图”

规划算法算出来的不是一句 “我觉得能走”，而是一份带来源、依赖和使用条件的不可变工件。

TransferPlan		不可变
来源快照	world:0	
依赖版本	drop-zone=0 · map=0 · robot=0	
路线内容	起点姿态 → 中间路径点 → 目标姿态	
安全信息	占用指纹、机器人模型、载荷约束、校验策略	
消费者	TransferHeld physical skill	



旧施工图不能被偷偷涂改

如果现场变了，就保留 v1，重新生成 v2，并记录为什么换图。

- 保留旧工件**

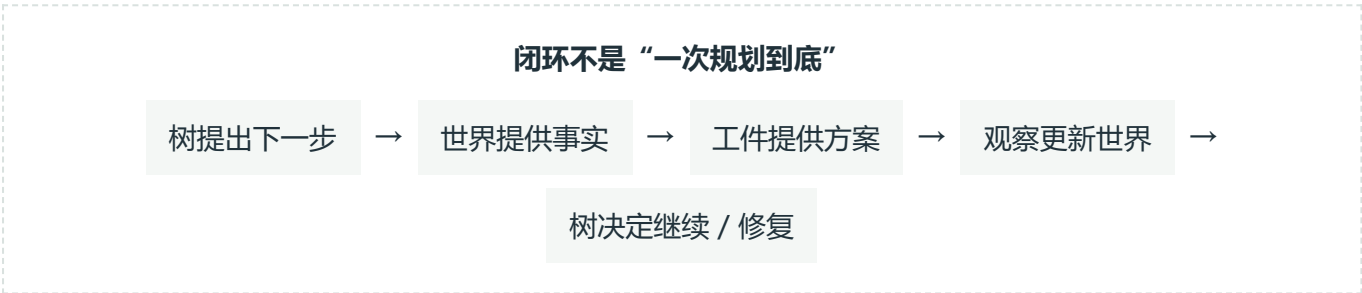
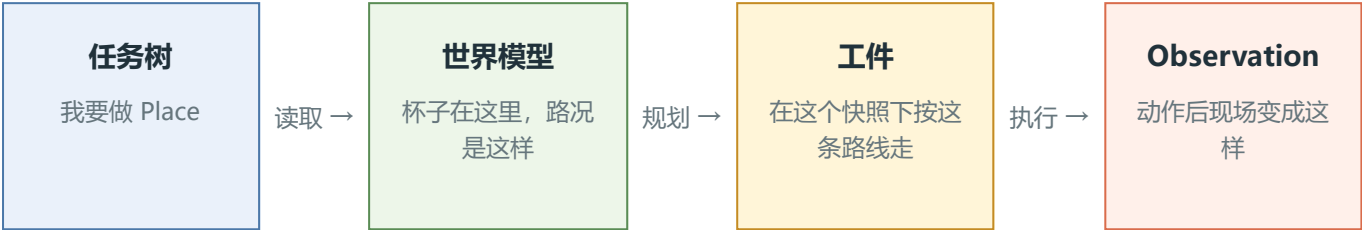
便于审计、回放和解释。
- 发布新工件**

让后续动作使用与当前现场匹配的方案。
- 按依赖判断新鲜度**

不是随意设置一个 “过期” 布尔值。

三个核心对象如何连起来？

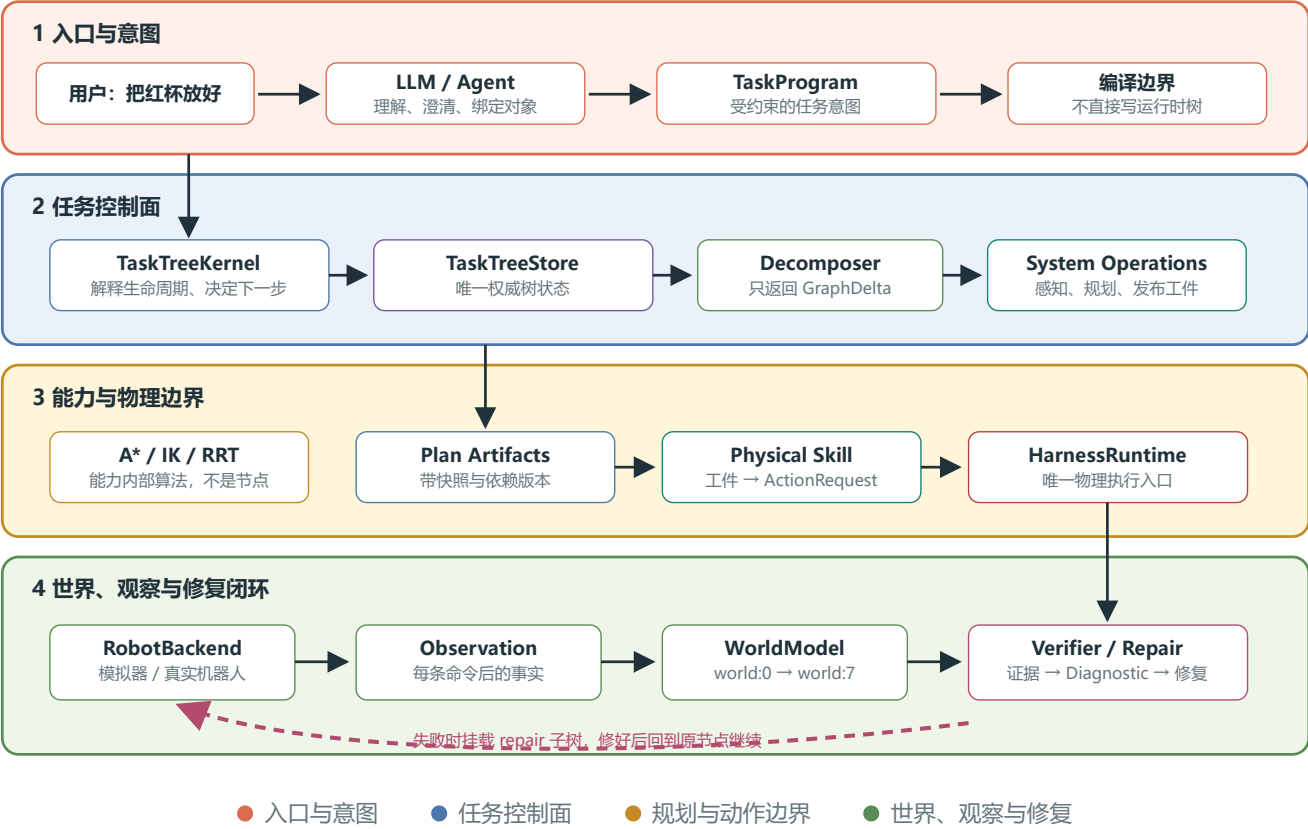
系统不是先凭空写一棵大树，再把它强行执行；而是让树、世界和工件互相校验。



用一句话说
树决定“做什么”，工件说明“怎么做”，观察证明“做完后真的发生了什么”。

全局架构图：从输入一直走到物理世界

下面这张图是整套系统的“地图”。先看四层，再看箭头，不要一上来背模块名。



看箭头的顺序

实线是正常推进：输入 → 编译 → Kernel → 工件 → Runtime → 机器人；虚线是反馈：Observation → WorldModel → 验证 / 修复 → 回到 Kernel。

入口层负责“收任务”，不负责“偷偷做动作”

浏览器、语音、HTTP 和连续会话都只是入口适配器，它们不拥有运行时树的写权限。



入口可以做

- 提交对象和目标标识。
- 读取世界、树、事件和历史。
- 请求完整场景重置。
- 显示任务进度和最终结果。

入口不能做

- 直接修改运行时节点。
- 直接挂载 repair 子树。
- 直接调用规划器或机器人后端。
- 把多个根目标硬塞进同一棵树。

当前实现的真实边界

连续会话目前是串行执行：每个任务有自己的 Store / Kernel / 树，但共享同一个 WorldModel、Runtime、事务账本和机器人后端。

LLM 能做什么，不能做什么？

这不是削弱模型，而是把模型放在它最擅长、也最容易被约束的位置。

LLM 可以做

- 理解自然语言目标。
- 向用户追问缺失信息。
- 把对象和目标绑定成明确标识。
- 提交规范化的 TaskProgram。
- 提出高层任务意图。

LLM 不可以做

- 直接修改 TaskTreeStore。
- 直接调用 RobotBackend。
- 伪造 Observation 或验证结果。
- 跳过 Runtime 的安全、租约和事务。
- 把内部算法迭代伪装成业务节点。

权限

模型提交“申请单”，系统决定“能不能执行”。

如果模型给出了一个不合法任务，系统应该拒绝或要求澄清，而不是因为“模型很聪明”就放宽边界。可靠性来自约束，不来自祈祷。

编译器和 GraphDelta：为什么不允许随便改树？

分解器不是拿到一把“改数据库”的钥匙，而是提交一份受约束的结构增量。



- 只能把 child 挂在当前展开节点下面。

repair 必须恰好从失败节点挂一条 repair 边。
- 新节点不能跑出声明的子树范围。

完整图必须无环、可达、可解释。

为什么叫 Delta?

因为它只描述“增加什么结构”，而不是让外部组件直接改掉整棵树。这样 Kernel 仍然是唯一的解释者和写入者。

Kernel 与 Store：一个负责解释，一个负责保存

这是整个架构最重要的所有权边界。谁能写状态，决定了系统能不能被控制。



如果每个模块都能写树，会发生什么？

一个模块标记 succeeded，另一个模块又改成 failed；修复分支没有父节点；执行记录 and 实际状态互相打架。

三类节点：分解、系统、物理，各自只能拿到一部分权限

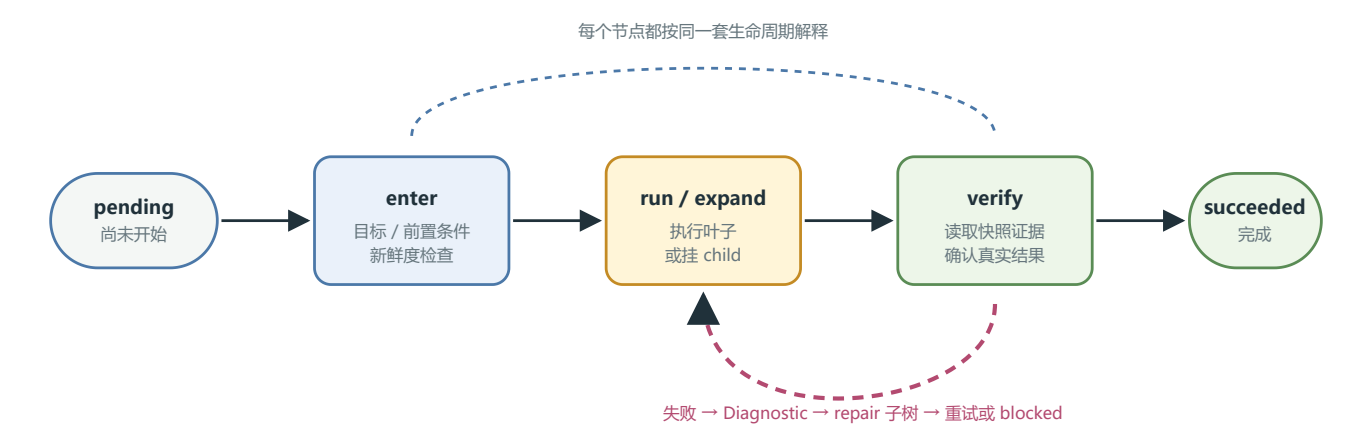
节点类型不是名字游戏，而是“它能看到什么、能改变什么”的运行时契约。

D Decomposer 把一个复合任务拆成 child 节点。 能做： 返回 GraphDelta 不能做： 调机器人、直接写 Store	S System Operation 读世界、做选择、做规划、发布工件、做验证。 能做： 读 WorldSnapshot、调用能力 不能做： 直接触碰 Backend	P Physical Skill 把已经批准的工件翻译成动作请求。 能做： 读声明的 Artifact 不能做： 绕过 Runtime 执行
--	---	--

A*、IK、RRT 放在哪里？	它们是能力内部的算法。算法迭代次数不应该变成任务树节点，否则树会记录大量没有业务意义的内部细节。
---------------------------------------	---

一个节点从 pending 到 succeeded, 究竟经历了什么?

所有节点都经过同一个 Kernel 生命周期。复合节点可以展开，叶子节点可以执行，但最后都要验证。



进入 读取目标和当前状态，检查是否已经满足。	前置 确认工件新鲜、条件成立、资源可用。	执行 / 展开 叶子做工作，复合节点返回 GraphDelta。	后置 根据不可变快照和证据判断结果。	失败 生成 Diagnostic，可能挂 repair，也可能进入 blocked。
----------------------------------	--------------------------------	--	------------------------------	--

一个容易误解的细节

某个目标已经满足时，叶子节点可以跳过动作；但结构节点不能因为目标满足就跳过必要的展开，否则树形结构和后置验证会失去意义。

案例一：正常放置任务的完整任务递归树

这是“没有故障”的参考执行。树里记录的是业务步骤，不是 A* 或控制器的每一轮计算。

16 最终节点数 1 个根 + 15 条 child 边对应的节点关系	15 child 边 正常分解关系	0 repair 边 本次没有故障修复	5 物理事务 5 个物理节点	127 事件数 可回放的执行证据
--	--	--	---	---



读这棵树

先看根节点 Place，再按从上到下的顺序读。缩进表示子任务；绿色 succeeded 表示后置验证已经通过，而不只是“函数返回了”。

案例一：正常任务按步骤走一遍

现在不要看代码，只把它当成一个现场工作人员的操作流程。

1

ResolveAndInspect

确认 cup-red 和 drop-zone 确实存在，建立对象绑定。

产出 BindingArtifact

2

PlanPick

规划机器人如何到杯子旁边，以及如何抓取。

产出 NavigationPlan + PickPlan

3

PlanTransfer

在还没拿杯子时，先规划带着杯子去目标区的路线。

产出 TransferPlan

4

Pick

到抓取位置 → 执行抓取 → 观察确认杯子确实被夹住。

3 个子节点

5

TransferHeld

切换携物姿态 → 带着杯子走 → 确认可以释放。

3 个子节点

6

Release

执行释放 → 观察确认夹爪松开且杯子不再被持有。

2 个子节点

7

VerifyPlaceGoal

检查杯子最终位置落在 drop-zone 接受范围内。

最终结论由证据得出

案例一：为什么一次成功不是 “一次 API 调用成功” ？

参考数据里，一次正常 Place 任务有 5 次物理事务、127 个事件，并把世界从 world:0 推进到 world:7。

world:0 起始现场	T1 NavigateToPickStance 观察后 world:1	T2 ExecuteGrasp 观察后 world:2	T3 MoveToTransportPosture 观察后 world:3	T4 NavigateHeld 观察后 world:4	T5 ExecuteRelease 观察后 world:7	world:7 目标已达成
------------------------	--	--	--	--	--	-------------------------

事件 127 包括进栈、状态变化、GraphDelta、出栈等	物理事务 5 每个事务都有请求 ID、检查、执行、观察	节点 16 业务可读、可定位、可回放
---	---	--

为什么事件比物理事务多？

因为一个物理动作前后还要记录节点进入、状态变化、执行栈和观察处理。多出的记录不是噪音，而是为了知道 “系统是怎样走到这个结果的” 。

一个物理动作，要过六道门才算完成

这六道门把“计划”变成“受控副作用”。任何一门不通过，都不能假装动作已经发生。



物理技能只负责构造 `ActionRequest`；真正的检查、租约、事务、执行、观察和对账由 `HarnessRuntime` 统一完成。

为什么每条命令后都必须有 Observation?

因为命令是“请求”，观察才是“事实”。系统不能把两者混为一谈。



为什么计划会过期？ 因为世界会变

同一条路线在 world:0 可行，不代表到了 world:7 还应该照着执行。



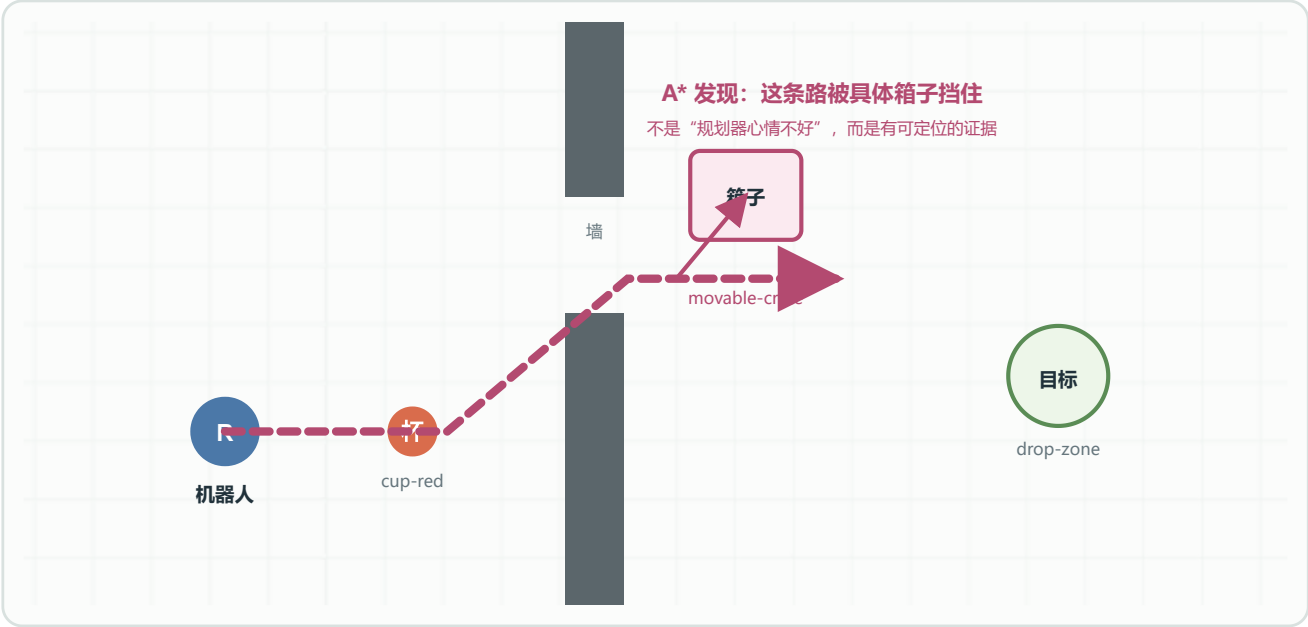
工件记录	当前世界比较什么	不一致意味着
source snapshot	快照引用	不能把旧现场当新现场
entity / map version	实体和地图版本	障碍或目标可能变化
robot state epoch	机器人状态纪元	轨迹起点可能不再连续
model / policy fingerprint	模型与安全策略	原计划不满足当前规则

不可变的意义

旧工件仍然可以被审计；新工件负责继续执行。系统不是把历史擦掉，而是把“为什么换计划”留下来。

案例二：路线被箱子挡住了

故障不是“机器人突然不聪明”，而是规划器发现了一个可以描述、可以验证的现实条件。



故障类型	ROUTE_BLOCKED	阻碍物	movable-crate，可移动，且被 A* 识别为具体 witness
修复前提	移走这个箱子后，反事实 A* 能证明路线恢复；并且存在可达 parking-zone。	当前状态	还没有拿起杯子，夹爪是空的，因此可以安全先处理箱子。

为什么不能简单地“再试一次”？

如果现场条件没有改变，重试只会重复同一个失败；如果现场已经部分改变，盲目重试还可能造成二次副作用。

做法	表面上发生什么	实际风险
原计划再跑一次	继续撞同一条被挡的路线	没有改变失败原因
让 A* 偷偷搬箱子	路线算法内部产生物理副作用	任务树看不到搬箱子，无法审计和验证
让模型自由发挥	可能临时调用未授权动作	越过安全边界，难以回放
结构化修复	先挂 repair 子树搬箱子，再刷新依赖工件	原因、动作、结果都可追踪

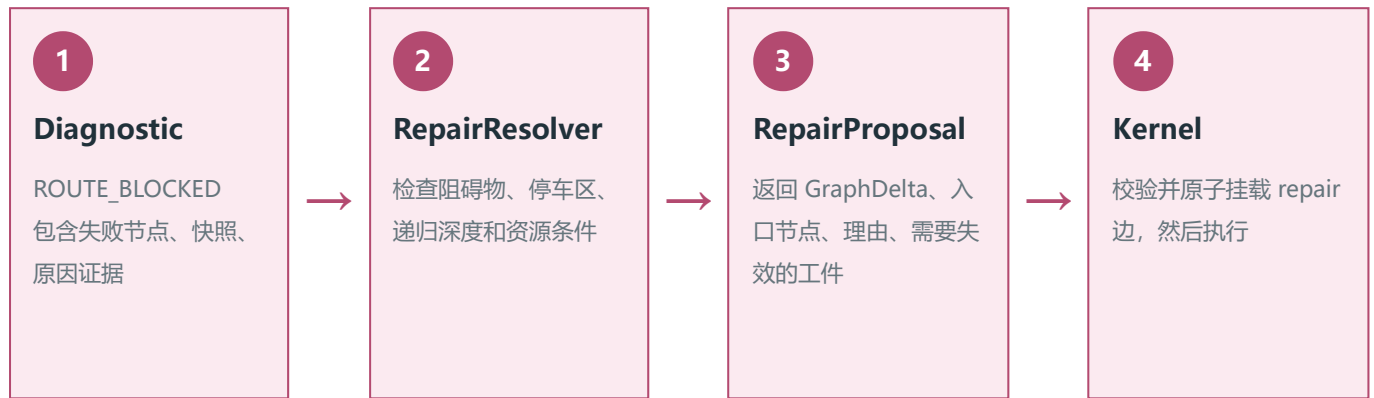
规则：算法可以证明“移动某障碍后会有路”，但不能在算法内部偷偷移动障碍。物理副作用必须成为任务树里显式的工作。

更直白地说

如果系统搬了东西，却没有在任务树里留下“谁搬的、为什么搬、搬到哪里、是否验证成功”，那它就无法可靠地解释和恢复。

故障如何从一句报错，变成一份可执行的修复方案？

Repair 不是一个隐藏的 catch 分支，而是一个结构化闭环。



Resolver 能做

- 读世界和工件。
- 分析故障原因。
- 提出修复子树。
- 说明理由和影响范围。

Resolver 不能做

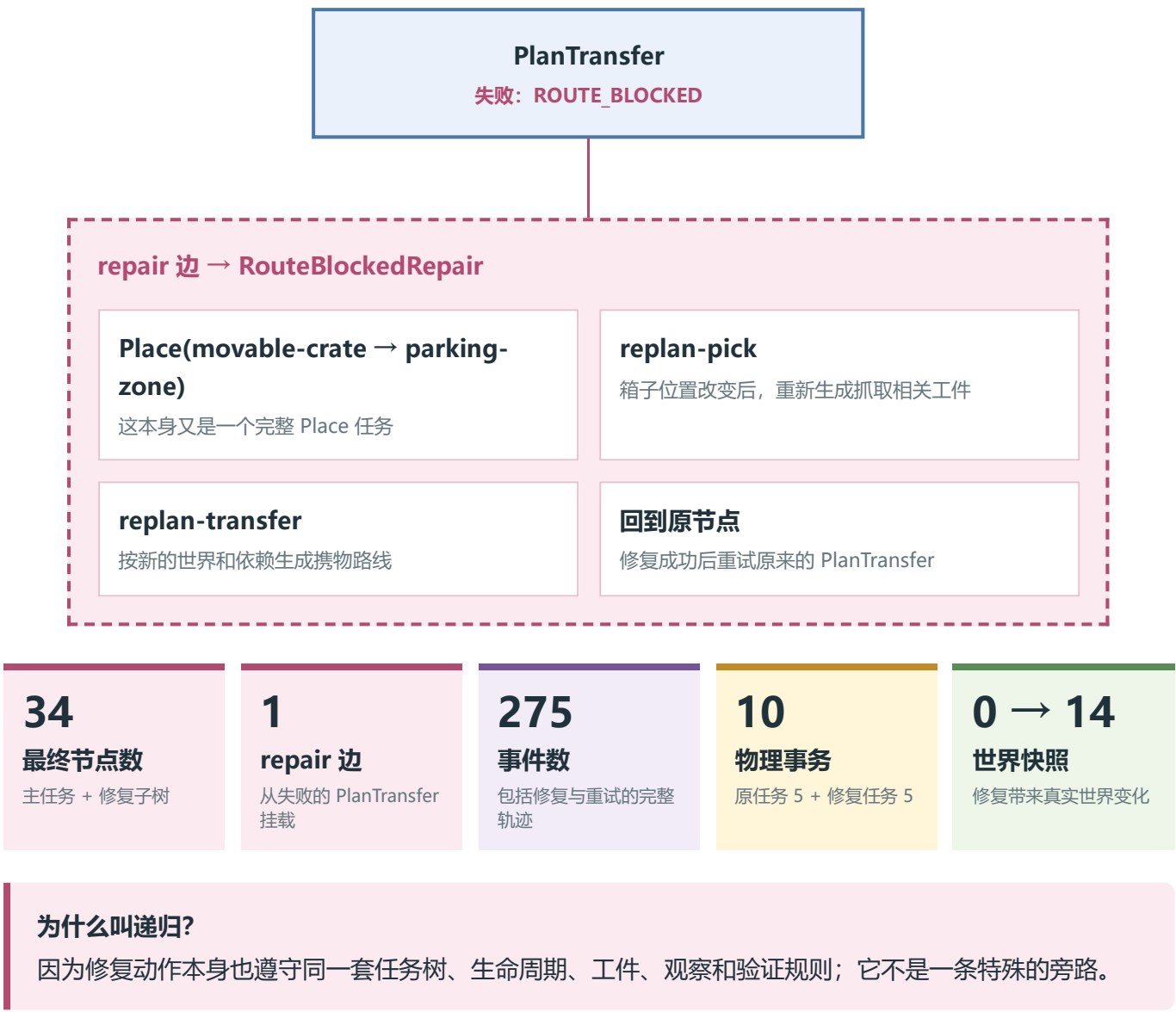
- 直接改 Store。
- 直接执行机器人动作。
- 自己宣布修复成功。
- 绕过 Kernel 挂边。

成功由谁判断？

不是 RepairResolver 说“我修好了”，而是 repair 子树执行后，由新的 Observation 和 Verifier 判断修复是否真的成功。

案例二：递归修复树到底长什么样？

修复任务不是漂浮在主流程外面的脚本，而是从失败节点下面显式挂出的一棵子树。



案例二：一次路线故障的详细执行过程

按时间顺序看，系统不是放弃原目标，而是先改变阻碍目标的现场条件。

1

PlanTransfer

在 world:0 规划携物路线，发现路线被 movable-crate 挡住。

产生 ROUTE_BLOCKED

2

反事实 A*

假设只移走这个箱子，重新计算后路线恢复；证明它是可修复 witness。

不是随便挑一个障碍物

3

挂载 repair

Kernel 校验 RepairProposal，并从失败的 PlanTransfer 挂一条 repair 边。

记录 rationale 与影响范围

4

递归 Place 箱子

对 movable-crate 执行完整的 Place：规划、抓取、携带、释放、验证。

世界推进到 world:7

5

刷新工件

箱子位置已经变化，原依赖可能过期；重新生成 PickPlan / TransferPlan。

新工件基于新快照

6

重试原节点

回到原来的 PlanTransfer，在新现场条件下重新执行。

不是换了用户目标

7

最终 Verify

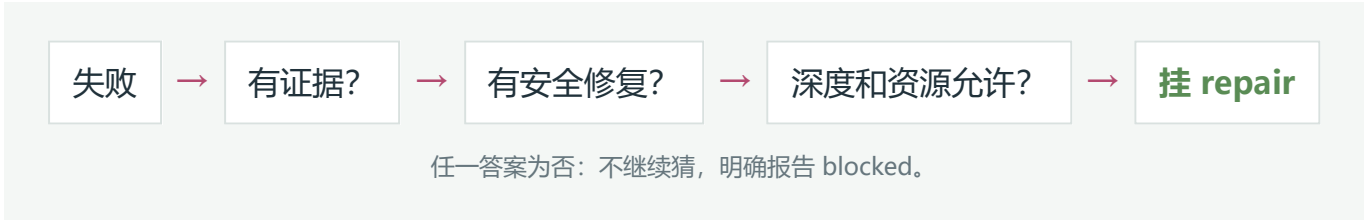
杯子到达 drop-zone，原 Place 根节点 succeeded。

世界最终到 world:14

递归修复会不会无限套娃？不会，系统有明确的收敛条件

“能修复”不等于“永远修复”。真正的工程系统必须知道什么时候停止。

递归深度上限 修复任务带有 repair_depth，超过上限就不能继续挂新的修复。	证据门槛 没有具体 movable witness、可行停车区和反事实路径证明，就不能提出搬障碍修复。
资源占用限制 如果机器人已经拿着杯子，不能再去抓第二个障碍物；应改为重新规划携物路线。	失败收敛 重复失败、无安全绕行、无可达停车区时，进入 blocked，交给人工或上层策略。



工程上的诚实

一个明确的 blocked，比系统不断尝试未知动作更有价值。它让人知道需要补充能力、改变现场，还是由人工接管。

连续多任务：树隔离，物理世界连续

这是整套架构很容易被误解的一点：每条指令有自己的树，但下一条指令看到的是上一条执行后的真实现场。



每个任务独立	跨任务共享
TaskTreeStore / Kernel / Inspector	WorldModel / HarnessRuntime / Backend
根节点、执行栈、事件和任务目标	机器人真实状态、实体位置、事务账本
防止两条根目标混在一棵树里	保证第二条指令看到第一条的结果

当前实现的准确描述

这是串行连续会话，不是已经完成的并行调度系统。若未来需要并行，必须另加资源调度、冲突仲裁和多任务一致性设计。

安全与治理：不是一句提示词，而是一串硬边界

物理动作的安全性来自多个层次共同作用，而不是寄希望于模型每次都理解对。

意图边界	LLM 只能提交规范化 TaskProgram
树边界	Kernel 唯一解释生命周期，Store 唯一权威状态
工件边界	计划带快照、依赖版本和安全策略指纹
执行边界	Runtime 统一做 freshness、guards、leases、checkpoint、事务
证据边界	每条命令后收 Observation，Verifier 只读快照
修复边界	失败变 Diagnostic，修复以 repair 子树显式出现

真实机器人和模拟器的差异

模拟器可以回滚内存；真实后端通常不能物理回滚。因此生产适配器应报告 `supports_rollback = False`，依靠 Observation reconciliation 对账，而不是假装动作被撤销。

高层设计原则：为什么这套架构值得做？

优势不在于模块数量更多，而在于把隐含行为变成可检查的契约。

单一所有权	一个 Store、一个 Kernel 写入者，减少状态冲突。	职责隔离	意图、规划、物理动作、验证分别由不同角色承担。
证据驱动	成功来自 Observation 和快照证据，不来自函数返回值。	工件不可变	历史不被偷偷修改，新现场发布新计划。
失败显式化	Diagnostic 和 repair 边让恢复进入正常工作流。	任务 / 世界分层	每条任务树可隔离，真实现场可连续复用。

管理层看到的价值	技术机制	可以观察的证据
风险可控	角色权限 + Runtime 唯一物理入口	ActionRequest、事务、Observation
故障可恢复	Diagnostic + repair subtree + 依赖重规划	repair 边、重试次数、最终状态
过程可解释	任务树、执行栈、事件日志、世界快照	127 / 275 个事件可回放
能力可替换	任务与算法、机器人通过协议隔离	替换 decomposer / backend 的契约测试

把技术优势翻译成老板听得懂的业务价值

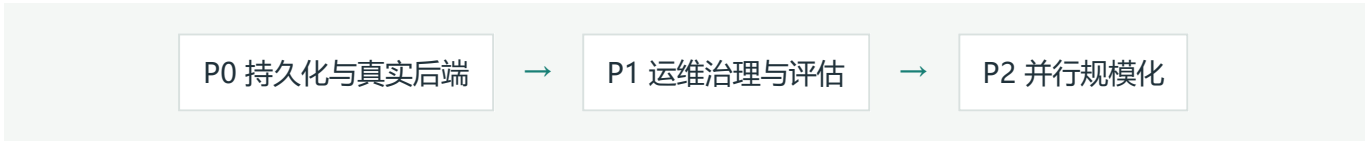
这里不把参考实现的演示数字夸成生产 SLA，而是说明它能支持什么样的管理决策。

少一点“玄学失败” 失败会落到具体节点、具体诊断和具体证据，减少“模型刚才不知道怎么了”。		少一点整条流程重做 障碍物只影响局部计划时，可以局部修复和局部重规划。	
更容易换能力 A*、IK、RRT、机器人后端可以替换，任务语义和控制面保持稳定。		更容易做试点验收 可以定义成功率、恢复收敛率、人工介入率、过期计划拦截率和回放完整率。	
任务成功率 最终 Verify 是否通过	恢复收敛率 发生故障后最终成功的比例	人工介入率 进入 blocked 后需要人处理的比例	回放完整率 任务、事件、工件、快照能否对齐
决策提醒 参考案例里的 16 / 34 节点、127 / 275 事件，是架构证据，不是对未来生产吞吐量或成功率的承诺。生产验收要用真实设备和真实任务重新测量。			

当前参考实现，离生产系统还差什么？

架构契约已经明确，但参考实现仍然有清楚的边界。把边界讲出来，反而更专业。

当前参考实现	生产化需要补齐	优先级
Store 主要是内存实现	树、执行栈、事件、别名、工件需要同一事务持久化	P0
JSON 用于检查和导出	需要完整崩溃恢复协议和幂等重放	P0
确定性的平面双连杆模拟器	真实机器人适配、传感器噪声、控制周期和安全验收	P0
本地单操作员控制台	认证、权限、多操作员审计和部署隔离	P1
串行连续会话	如需并行，增加调度、资源仲裁和冲突解决	P1
保守网格 / 圆形碰撞模型	接入生产级碰撞库、附着载荷模型和在线监控	P1



未来增加新任务、新机器人、新算法，应该怎么接？

扩展点应该落在协议上，而不是把所有逻辑重新塞进 Kernel。

新增高层任务

- 增加已校验的 TaskProgram action。
- 只编译根节点。
- 注册 decomposer，返回 GraphDelta。
- 复用已有 system operation 和 physical skill。
- 补端到端测试和树形状断言。

新增机器人

- 实现 RobotBackend protocol。
- 把厂商 SDK 类型封装在 adapter 内。
- 把 telemetry 转成 Observation。
- 不把厂商私有状态暴露给 planner 或 verifier。
- 保留 ActionRequest / reconciliation 契约。

新增 MPC

- 实现 JointTrajectoryController。
- 注入 backend / controller adapter。
- 保持 ActionRequest 和 Observation 不变。
- 不要把 MPC 迭代变成任务树节点。
- 用安全边界和回放测试验收。

判断标准：新增能力应该改变“局部实现”，而不是破坏“任务树、工件、Runtime、Observation”的公共契约。

术语表：把技术词翻译成成人话

看到这些词时，先用右边的人话理解，再回头看左边的英文。

TaskProgram	模型提交的结构化任务意图，像标准订单。
TaskTree	会执行、会记录状态、会挂修复的任务树。
TaskTreeKernel	任务树的执行控制内核，决定下一步怎么推进。
TaskTreeStore	运行时树状态的唯一权威账本。
WorldModel	机器人和现场事实的版本化账本。
WorldSnapshot	某一时刻的不可变现场照片。
Artifact	基于某个快照生成的不可变施工图。
GraphDelta	受约束的树结构增量，不是随便改数据库。
Diagnostic	结构化失败原因，例如 ROUTE_BLOCKED。
RepairProposal	修复器提出的修复子树方案。
ActionRequest	物理技能生成的受限动作请求。
Observation	动作后观察到的真实证据。
reconciliation	无法物理回滚时，根据观察重新对账事实。
freshness	工件依赖是否仍与当前世界一致。

一行速记

TaskProgram 是订单，TaskTree 是施工计划，Artifact 是施工图，ActionRequest 是派工单，Observation 是现场照片。

想继续往代码里看：模块分别在哪里？

这页不要求非技术听众记住，只用于让 CTO 能快速把图和代码对上。

服务入口	<code>src/task_recursive_tree/integrations/gemini_er2/server.py</code> 浏览器 / HTTP / 连续会话入口
编译桥	<code>src/task_recursive_tree/integrations/gemini_er2/compiler.py</code> Harness DTO → Kernel 定义
执行桥	<code>src/task_recursive_tree/integrations/gemini_er2/executor.py</code> 装配 Store、Kernel、适配器
任务内核	<code>src/task_recursive_tree/task/kernel.py</code> 唯一生命周期解释器
权威 Store	<code>src/task_recursive_tree/task/store.py</code> 节点、运行态、边、栈、事件
节点模型	<code>src/task_recursive_tree/task/model.py</code> Node、Edge、GraphDelta
系统操作	<code>src/task_recursive_tree/integrations/gemini_er2/operations.py</code> 感知、选择、规划、验证
物理技能	<code>src/task_recursive_tree/integrations/gemini_er2/skill.py</code> Artifact → ActionRequest
物理运行时	<code>src/task_recursive_tree/integrations/gemini_er2/physical_runtime.py</code> 唯一后端执行边界
语义与修复	<code>src/task_recursive_tree/integrations/gemini_er2/semantics.py</code> / <code>repair.py</code> 验证、Diagnostic、RepairProposal

本 PDF 依据项目根目录的 ARCHITECTURE.md、README.md、中文架构说明和 .artifacts/ppt-examples 中的执行证据编写。参考数据用于解释架构，不代表生产 SLA。

最后只记住这张图：四句话讲完整套架构

如果你要向别人复述，按照下面四句讲，基本不会跑偏。

第一句：模型

LLM 负责理解人话、澄清目标、提交受约束的 TaskProgram。

第二句：控制

Kernel 负责解释任务树，Store 保存唯一权威状态，决定下一步怎么推进。

第三句：执行

系统基于世界快照生成不可变工件，物理技能把工件变成 ActionRequest，Runtime 才能碰机器人。

第四句：恢复

每条动作后收 Observation；失败变 Diagnostic，修复以 repair 子树进入同一套执行流程。



最终判断标准

不要问“模型看起来聪不聪明”，要问：任务是否可解释、动作是否受控、结果是否有证据、失败是否能收敛。

Task Recursive Tree Agent · 白话详解版